Harness Design Brief — StoreOps

Track: Build (Solution Architect) · **Stack:** Java 17, Spring Boot 3.x, Maven, JUnit 5 + MockMvc, Checkstyle, SpotBugs, JaCoCo, ArchUnit · **Feature:** shift handover bulk update, `PATCH /api/activities/bulk-status`

Section A — Intent Decomposition

Outgoing shift staff need to close out a shift in one action. Today they PATCH each activity individually — on a handheld at shift end, twenty taps and a half-closed shift when someone gives up. The feature takes a list of activity ids with target status `DONE` or `BLOCKED`, applies what it can, reports what it cannot, writes an audit entry per successful transition, and notifies the incoming shift's department lead. Its likeliest defect is failure mode 3 in its natural habitat: a test asserting `200 OK` without checking which items actually changed.

Sprint boundaries

Boundaries sit at **risk seams, not layer seams** — a FAIL on one side must not force a rewrite of passing work on the other.

Sprint	Delivers	Why the boundary is here
1	Service-layer bulk transition: per-item outcome, transition validation, audit entry per success	The only hard part, fully testable without HTTP. Nothing depends on it yet, so a rewrite is cheap.
2	web layer: DTO with Bean Validation, controller, 200/207 mapping, MockMvc tests	Depends only on sprint 1's result type. Separated so a DTO rename never churns validated service logic.
3	`ShiftHandoverCompletedEvent` published once per operation; `alerts` listener creating the notification	ARCH-2 is at maximum risk — "notify the department lead" is where a model reaches for `NotificationService`. Isolating it means an ARCH-2 FAIL retries ~40 lines, not the feature.

Deliberately not a sprint: testing. Tests ship with the sprint whose behaviour they prove, because gate `AC-1` requires the evidence to exist when the verdict is written.

What makes a criterion testable

Every THEN clause names an observable — a status plus a named body field, a persisted field value, an `AppException` subclass and code, an event type and payload field, or a count. The test is whether the criterion can fail: "errors are handled gracefully" cannot, because no state of the world contradicts it. A subjective AC does not merely score badly, it makes the verdict non-deterministic, since two readings of the same code are both defensible.

Negative clauses are mandatory wherever a rule says something should *not* happen. The characteristic bulk-endpoint bug is doing the work *and also* something extra, like writing an audit entry for a rejected item.

Example contract entry, in full

```
AC-1.2
GIVEN an activity with status DONE
WHEN bulkUpdateStatus is called with that activity id and target status BLOCKED
THEN the result's failed[] contains an entry for that id with reason INVALID_TRANSITION
AND the activity's status in the repository is still DONE
AND no audit entry is created for that id
AND the result's succeeded[] does not contain that id
```

Four clauses, four observables, one test. Clauses two and three are the negative assertions — without them the criterion is satisfied by code that correctly reports the failure and persists the change anyway. Cited against `BulkStatusServiceTest#rejectsTransitionFromDone`; gate `AC-1` turns on whether that method asserts all four.

Section B — Governance Framework

Skill file strategy

Eight files, split by **who reads them** rather than by subject, because an agent that reads a file is influenced by it.

File	Read by	StoreOps-specific content
`app-context`	all four	Five modules, entities and enums, the ownership matrix, the four client failure modes
`architecture-principles`	Planner, Generator, Evaluator	ARCH-1..4 and ERR-1, each with its ArchUnit rule and what breaks without it
`sprint-decomposition`	Planner	Risk-seam boundary rules, the observable-naming AC rule
`coding-conventions`	Generator	`storeops.<module>.{web,service,repository,domain}` layout, the `AppException` subclass/status table
`api-integration`	Generator	The 207 partial-success envelope, the 400/422 split, `StoreOpsEventBus`, the five-

		event catalogue, `StaffRole` scoping
`how-to-test`	Generator	Test type per layer, the four coverage thresholds, what a behavioural assertion is
`how-to-review`	Evaluator	Review order, standard of evidence, what ArchUnit cannot prove
`evaluation-criteria`	Evaluator	Four dimensions, eleven gates, scoring arithmetic, verdict thresholds

Two are shared, for different reasons. `app-context` because domain facts must be identical everywhere — a Planner and Evaluator disagreeing about whether `reports` may write is a governance failure, not a difference of opinion. `architecture-principles` because Generator and Evaluator must read the *same wording*: if the Generator's ARCH-2 is looser, the loop oscillates, with the Generator writing what its file permits and the Evaluator failing what its file forbids.

Two exclusions are as deliberate. The Evaluator does not read `coding-conventions` — an assessor holding a style guide mines it for findings, and style is not a gate. The Generator does not read `evaluation-criteria` — an implementer that can read the rubric optimises for the rubric, and the gap between rubric and contract is the signal the Evaluator exists to detect.

The reviews archive as audit trail

`.harness/reviews/` holds, per sprint, the contract, generator summary, evaluator feedback and run log. Committed, and **append-only** — a superseded verdict gets a new file, never an edit, because an audit trail that can be corrected in place is a narrative rather than a record. Access is by repository read, which was a design goal: the standards team withholding rollout approval can audit it without asking permission.

How it surfaces a recurring issue: the Monitor's `## Skill file signal` fires when the same gate ID fails on the *first* iteration of three or more consecutive sprints. If `ARCH-2` fails first-iteration on sprints 4, 5 and 6 and passes by iteration 2 each time, no sprint looks bad — all three end PASS. The pattern says the Generator is not being told something clearly enough *before* it acts, because it reliably fixes the same thing after being told. The fix is upstream in `api-integration`, not downstream in a stricter Evaluator. Without the archive it is invisible, since each verdict is read once and forgotten.

One rule, traced

Gate ARCH-2: *any state change whose effect lands in another module is raised via `StoreOpsEventBus.publish(...)`; a module never injects, imports or calls another module's service to cause a change. Reads are exempt; writes are not.* Proven by `ArchitectureRulesTest#alertsIsReachedByEventOnly` — no class outside `com.cognizant.storeops.alerts..` may depend on `...alerts.service...`

What breaks without it. Side effects become unenumerable. "What happens when a shift handover completes?" stops being "an event is published and these listeners consume it" and becomes "whatever the services that happen to import each other do" — failure mode 4 verbatim.

It must be a gate rather than a convention because the direct call is shorter and better documented: any model asked to notify a department lead reaches for `NotificationService.create(...)` first, since that is what the training distribution contains, and a convention contradicting the likely generation is violated every sprint. The rule also constrains *how* the bus is used — inject `StoreOpsEventBus`, never Spring's `ApplicationEventPublisher`, which is a legal idiom that routes around the type the gate inspects.

Section C — Non-Determinism Strategy

Dimensions and weights

Dimension	Weight	Reasoning for StoreOps
Architectural compliance	40%	Three of four client failure modes are architectural. What the standards team is withholding approval over, and what human review is worst at — a boundary violation reads as correct code.
Acceptance criteria satisfaction	25%	Below architecture deliberately: a correct feature violating a boundary is worse than an incomplete one respecting it. The violation ships silently and compounds; the gap is visible and gets scheduled.
Test quality and coverage	20%	Failure mode 3. Separate from acceptance because an AC can be satisfied by code whose test proves nothing — collapsing them lets a green suite carry the acceptance score.
Build integrity	15%	Lowest weight, highest leverage. Nearly binary in practice, so it earns little score; its power is in its two gates, one of which protects all the others.

Hard gates, and why none can be soft

Eleven gates — nine decided by a tool exit code, two by reading a cited test.

Gate	Prevents	Why not advisory

ARCH-1 cross-module repository imports	Mode 1	Compiles, passes, behaves correctly the day it ships; no test catches it. Advisory means it ships.
ARCH-2 cross-module service writes	Mode 4	The violating code is the likely generation; a convention against it breaks every sprint.
ERR-1 raw exception throws	Mode 2	The Generator writing the throw also writes the expected status, so the suite agrees with the bug.
ARCH-3 layer direction	—	Logic duplicates across controller and service, drifts, behaviour differs per route.
ARCH-4 reports read-only	—	A report run mutates what the next run reads; the trail is gone before it surfaces.
AC-1 every AC verified	—	The only link between verdict and contract; without it the loop terminates on a feature never built.
AC-2 no out-of-scope files	—	Scope creep accumulates invisibly; each diff looks locally reasonable.
TEST-1 coverage thresholds	—	Coverage drifts down monotonically; no sprint looks like the one that broke it.
TEST-2 behavioural assertion per rule	Mode 3	A green suite over unverified rules is worse than a red one — it ends review.
BUILD-1 clean verify	—	A verdict on code that does not compile is meaningless.
BUILD-2 no new suppressions	all	The gate protecting the gates. Without it the rest are optional — one annotation and the harness is theatre.

Variable output to a deterministic verdict

Three mechanisms, in order. **Gates before scoring** — any gate failure is FAIL with no dimension scored, so most failing sprints are decided before an LLM assesses anything. **Binary checks, fixed arithmetic** — every check is binary and equally weighted within its dimension; `raw = (passed/total) × 100` truncated, `contribution = raw × weight`, one rounding at the end, no partial credit inside a check. **Fail closed on ambiguity** — an unresolvable check is recorded FAIL, prefixed `UNDETERMINED:`, with the required fix being to make the check decidable.

The `CONDITIONAL PASS` band (70–84) sits **entirely above the gates**; there is no path from a gate failure to a conditional pass. The band permitting discretion never touches the rules permitting none.

Worked example. Sprint 1, iteration 1: `clean verify -Pharness-gates` exits 0, ArchUnit clean, JaCoCo service 84% / overall 78%, no suppressions, all files in scope — ten of eleven gates decided on tool output alone. AC-1 needs the tests read: AC-1.2 names four observables and its cited `rejectsTransitionFromDone` asserts only that `failed()` is non-empty; AC-1.5's cited test asserts the succeeded count alone. **Verdict `FAIL`**, `HARD_GATES_FAILED: AC-1`, no score. Nothing in that chain depended on a judgement about whether the code was good — AC-1.2 names four observables, the test asserts one, and a second Evaluator reaches the same verdict because that is a count.

Iteration 2 adds the assertions. All gates pass and scoring runs:

```
D1 architectural  7/7 → 100 × 0.40 = 40.00
D2 acceptance     5/6 →  83 × 0.25 = 20.75   (one layer label wrong in the summary)
D3 test quality   7/8 →  87 × 0.20 = 17.40   (no negative assertion on the idempotency rule)
D4 build          5/5 → 100 × 0.15 = 15.00
                              total = 93.15 → 93  → PASS
```

Both shortfalls go to `## Advisory` — neither failed a gate, neither is worth an iteration against a budget of three.

Escalation

Trigger: the third consecutive FAIL on the same sprint — per sprint, not per run, because a sprint needing three attempts and a run needing nine are different problems.

Contains: sprint and feature; the gate IDs failing on all three iterations with the exact tool output; **what changed between iterations**, one line each; files touched; a `RECOMMENDED_ACTION` of `REVISE_CONTRACT` / `REVISE_SKILL_FILE` / `HUMAN_IMPLEMENT` with a reason; and a branch holding the last attempt. The what-changed field separates the Generator looping on one mistake, a skill file problem, from the Generator fixing one thing and breaking another, usually a contract asking for two incompatible things. Those need different human responses.

Recipient: the developer who invoked the run, copied to the engineering standards team when the failing gates include ARCH-1, ARCH-2 or ERR-1 — those exist for the standards team's own concerns, so a repeated failure is evidence their rule is unclear in the skill files or too strict. Their decision, not the developer's.

Expected response within one working day: amend the contract, amend a skill file, or implement by hand and log the gap. What must not happen is waiving a gate for one sprint — if a gate is wrong it is wrong for every sprint, and it changes in the open.

Section D — Architectural Decisions

D1 — Agents communicate only through files, never through conversation history

Decision. Each agent runs as a fresh invocation receiving its agent file, its declared skill files and its declared input artefacts — nothing else. The Generator sees `evaluator-feedback.md`, not the Evaluator's reasoning. Context resets at every sprint boundary.

Alternatives. (a) One shared conversation across all four agents — simplest, and how Claude Code is used by default. (b) A shared scratchpad each agent appends to. (c) Passing the previous agent's artefact *plus* its reasoning.

Rationale. Shared context degrades across three sprints with retries: by iteration 2 of sprint 3 the window holds two sprints of superseded code and the Generator starts defending decisions already rejected, which is the main cause of a repeated mistake. Architecturally, if agents influence each other outside the artefacts then the artefacts are no longer a complete record and `.harness/reviews/` is no longer an audit trail — which also rules out (b).

Assumption. That re-reading skill files each invocation is cheaper than carrying a degraded context — roughly, that the Generator's ~14k of skill files costs less than the 40k+ of accumulated conversation it replaces. True at current sizes; past ~25k for one agent the trade flips, and the answer is to split the agent, not to share context.

D2 — ArchUnit as the boundary gate, in the Maven test phase

Decision. ARCH-1..4 and the ERR-1 backstop are ArchUnit rules in `ArchitectureRulesTest`, running in the standard `verify` lifecycle under a `harness-gates` profile that CI activates unchanged.

Alternatives. (a) LLM-assessed review only. (b) `jdeps` or `maven-enforcer-plugin` dependency rules — the closest Java analogue to `dep-cruiser`. (c) A custom bytecode-scanning script. (d) Spring Modulith, which makes module boundaries first-class.

Rationale. (a) fails determinism outright: boundary violations are where an LLM assessor is most lenient, because the violating code reads as correct. (b) sits outside the test phase, so `mvn test` gives a false green and its failure names a JAR rather than a rule. (c) is bespoke tooling for a problem two libraries solve. (d) is technically the best answer, rejected on scope — adopting Modulith changes the *application's* architecture, and a harness governs a client's codebase rather than holding a licence to restructure it. ArchUnit runs in the phase CI runs, fails with a quotable named rule, and lives entirely in the test tree: throw the harness away and nothing in `src/main` changes.

Assumption. That boundaries stay expressible as package rules, `com.cognizant.storeops.<module>.<layer>`. A move to vertical-slice packaging breaks every rule here, and breaks them *silently* — the rules keep passing, they just stop meaning anything. Modulith would have failed loudly. This is the most likely thing to break this harness.

D3 — `CONDITIONAL PASS` exists, and sits strictly above the gates

Decision. `FAIL` on any gate failure or below 70; `CONDITIONAL PASS` at 70–84, advancing the sprint and appending open items to `debt-log.md`; `PASS` at 85+. A conditional pass requires all eleven gates to have passed.

Alternatives. (a) Binary PASS/FAIL. (b) A conditional pass that may waive a named non-architectural gate — letting TEST-1 slide two points. (c) A conditional pass requiring human sign-off before advancing.

Rationale. (a) was my starting position; the iteration budget moved me off it. A sprint can be architecturally sound and feature-complete with a thin test or a mislabelled layer, and retrying for that spends one of three attempts while risking a regression in correct code. (b) I rejected firmly: once a gate can be waived, every verdict asks whether *this* failure is waivable, which has no deterministic answer. (c) puts a human back into the loop the harness exists to automate.

Assumption. That `debt-log.md` is actually read. **This is the weakest assumption in the design** — nothing enforces it, and the Monitor records debt but cannot block on it. If the log goes unread, `CONDITIONAL PASS` degrades into a slow `PASS` and the 70–84 band becomes a quality leak. The fix I would build next is a Planner pre-check refusing to open a new feature while any debt item is more than N sprints old, moving enforcement to the one agent that runs before a human approves anything.